Algorhythms: Set Transformers for Playlist Recommendation

Matias Bronner
Brown University
matias_bronner@brown.edu

Brendan Rathier
Brown University
brendan_rathier@brown.edu

Daniel Schiffman
Brown University
daniel_schiffman@brown.edu

Camilo Tamayo-Rousseau
Brown University
camilo_tamayo-rousseau@brown.edu

***Abstract*—In this project, we attempt using deep learning to address the problem of playlist recommendation. Specifically, we build a system that produces five recommended songs given a playlist of songs with a title. We use a title-conditioned set transformer architecture with pooling by multi-head attention trained on user Spotify playlists with titles and tabular audio features about the songs in the playlists. We create a custom weighted approximate-rank pairwise (WARP) loss function that is in part a reconstruction loss and in part a more traditional WARP loss. While our model is able to train and improve on the metrics we use such as R-precision, our qualitative results are underwhelming and we see indications of mode collapse in our system's recommendations. We hypothesize that the title conditioning in our model is too weak and that the training data we use may be too noisy or even might not represent a function that is learnable.**

GitHub Repository: https://github.com/dschiff190/Algorhythms-DL-Playlist-Recommendation

I. Introduction

A. Motivation

Creating cohesive music playlists is a core part of the modern listening experience. Thus, providing automated recommendations to users to extend their playlists is a key feature for music-listening platforms like Spotify and Apple Music. To make this feature effective, the system must understand the "vibe" of a playlist. This task is challenging as the theme of some playlists are quite clear (e.g. "rap music playlist") whereas others have more complex themes that cannot be fully captured by simple features of the playlist such as genre or time period (e.g. "beach trip playlist"). As music platform users who have felt dissatisfied with playlist recommendation algorithms, we wanted to dive more deeply into how we could create a model that learns the "vibe" of a playlist to make our own recommendation system.

B. Overview

We built a title-conditioned playlist recommendation system that synthesizes both semantic information, which is captured in the playlist's title, and musical relationships, which are reflected in the Spotify-API features of user-selected songs. Given a playlist title (for example, "chill," "classic rock," or "study session") and a variably-sized sequence of songs the "user" has already added to their playlist, our model predicts five additional tracks that should be thematically appropriate for the playlist. Since playlists can be shuffle played, we treat them as sets which informs our architecture choice. We utilize set transformers with pooling by multi-head attention that allow us to leverage multi-head attention to learn vector representations of playlists without using transformers in a sequential manner. Additionally, we develop a customized version of weighted approximate-rank pairwise (WARP) loss that is partially a reconstruction loss.

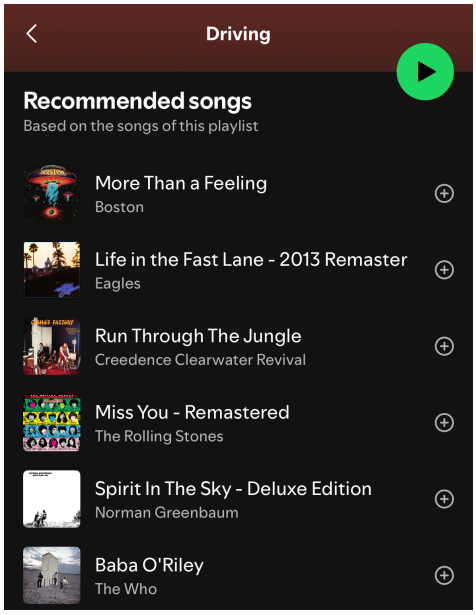

Fig. 1: Our goal was to create a similar system to the one seen here on Spotify that suggests five songs for a given playlist.

C. Goals

The following were our goals upon beginning the project.

- **Base Goal:** Implement a model that takes in a "user" playlist and can make five recommendations using our proposed architecture.
- **Target Goal:** Implement a recommendation system that appears to learn basic themes of playlists as measured by our metrics and qualitative evaluations (such as recommending in-genre songs to playlists organized by genre).
- **Stretch Goal:** Implement a recommendation system that appears to learn more complex themes of playlists as

measured by our metrics and qualitative evaluations (such as recommending applicable songs to playlists organized by more abstract themes - e.g. recommending upbeat/pop songs to a ”party” playlist).

II. Literature Review

Significant research has been done regarding song recommendation systems. However, the problem we address differs from what much of the prior work in the domain solves. In particular, our task does not rely on long-term user listening history or raw song audio, and it does not assume playlists are sequential. We have found several papers addressing playlist continuation in a way that is similar to our framing of the problem.

In particular, ”Automatic Playlist Continuation with a Hybrid Recommender System Combining Features from Text and Audio” [1] used a non-deep learning approach to the problem called Matrix Factorization. While this was shown to be efficient, its simplicity prevented it from capturing complex relationships. Another issue was that Ferraro et al. assumed that playlists are ordered sequences. They also incorporated audio data. We do not make the assumption of order in playlists and do not use audio files as data. However, the paper provided valuable metrics for the playlist continuation problem such as R-Precision which we describe in the Methodology section. Additionally, Ferraro et al. use a WARP loss which was the starting point for our custom loss.

”Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks” [2] was an inspiration for our architecture. We decided that playlists do not have a specific order, and thus a normal transformer would be a poor solution. When looking for a deep learning architecture that suited our needs, we came across this paper, which details a set transformer which is “permutationally invariant” but can still recognize complex relationships between songs. This paper gave us the Induced Set Attention Block and Pooling by Multi-head Attention which we use in our architecture.

III. Methodology

A. Dataset

For our project, we use the Million Playlist Dataset (MPD) from Kaggle, which contains 1 million user-generated Spotify playlists from 2010–2017 [3]. We join this playlist data with four additional Kaggle datasets of Spotify track-level audio features, each derived from the Spotify Web API [4], [5], [6], [7]. These song-level datasets provide features such as genre, tempo, popularity, danceability, duration, loudness, acousticness, instrumentalness, and more.

1) Size: After filtering and joining, our final playlist set contains 994,544 unique playlists. In its flattened form (where each row corresponds to a single playlist–track pair), the dataset contains approximately 36,677,934 rows.

2) Preprocessing: We first combine the four song-feature datasets into a single master_tracks table with one row per unique track_id. We standardize column names & enforce feature ranges based on the Spotify API specification. During this process, three features (year, popularity, explicit), are not available in all four datasets. To handle this, we convert each of these into two columns: feat_filled and feat_is_missing. The feat_is_missing column is a binary indicator (1.0 if the original value was missing, 0.0 otherwise), and the feat_filled column contains either the original value (if present) or the mean of that feature over non-missing entries.

After this, we min-max normalize all continuous numeric features to (0, 1). Next, we flatten the MPD playlist and perform a left join with master_tracks on track_id. Any playlist-track rows that have missing features are dropped entirely. The result is a single CSV where each row represents one track in a playlist along with its normalized audio features, genre one-hots, and optional-feature masks.

3) Train, Validation, and Test Split: This CSV is loaded as a DataFrame for modeling and grouped together by playlist id. We pad sequences of tracks within playlists, and we construct a token vocabulary from the K most frequent tokens/songs from the DataFrame, mapping all other tokens to UNK. In our final setup we use a vocabulary size of 30,000 tokens (songs). Finally, we serialize the processed dataset and split it into 80% train, 10% validation, and 10% test.

B. Model Architecture

Our data is formed in batches of playlist where each playlist is represented by a string title and a variable number of songs each with 96 features from the Spotify API.

For our architecture, we define a “playlist model” that utilizes a set transformer with multi-headed attention and pooling by multi-head attention among some other techniques. This playlist model includes first a pre-trained title encoder (the Universal Sentence Encoder from Google) [8] to encode the playlist title, a song encoder model (a one-layer MLP with 256 units) to encode the songs into a set embedding size, a set transformer that uses multi-headed self attention to gain context-enriched embeddings between the playlist title and songs in the playlist, pooling by multi-head attention that uses multi-headed attention and a seed vector to encode the playlist down to a set playlist dimension from which a final dense layer projects to the vocab size to make predictions.

More specifically, our set transformer is made of set attention blocks where the forward pass of one of these blocks includes normalization, multi-headed self attention with 4 heads, a dropout layer with dropout rate of 0.1, normalization, an MLP, and a dropout layer with dropout rate 0.1 with recurrent connections. Our set transformer is made of 3 of these set attention blocks. Using a set transformer is helpful because it allows us to leverage the power of multi-headed self attention in transformers without using positional encodings because our assumption is that playlists are not ordered. Our pooling by multi-headed attention has a forward pass where we learn one seed vector, then use multi-headed attention to have this seed vector attend to all the songs and titles in the playlist, a recurrent connection with dropout with a dropout rate of 0.1, and normalization to produce a final pooled vector. Using pooling by multi-headed attention is helpful because we

need a way to reduce our context-enriched playlist features into a set size and pooling by multi-headed attention allows us to learn the best way to do this reduction with attention rather than doing a more crude reduction like a simple average.

We also use appropriate masking throughout our model to account for the padding we use for variably sized playlists.

Thus, the forward pass of our playlist model works in the following way as depicted in Figure 2. First, the song features of the songs in a playlist are encoded to the embedding size by the song encoder. Second, the playlist title is encoded to the playlist embedding size by the universal sentence encoder (and a projection layer that projects the playlist encoding to the song embedding size) allowing the model to capture semantic information about the playlist title. Third, the encoded title and song encodings for the playlist are concatenated and passed into the set transformer which runs the features through the set attention blocks and then through the pooling by multi-headed attention to produce a final vector representing the playlist of the playlist embedding size. Finally, this vector is passed into a final dense layer to project to the vocab size to get the logits of the prediction.

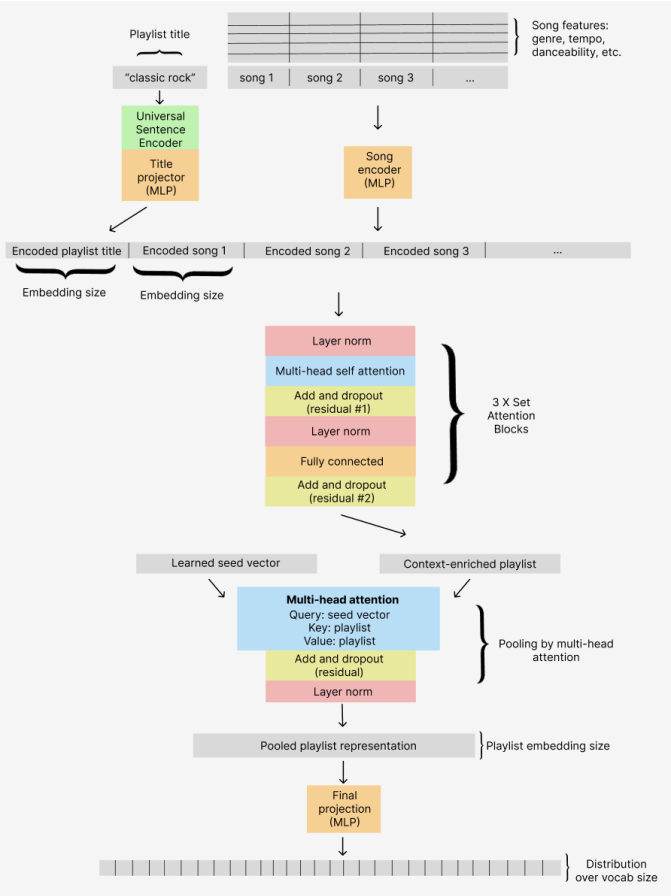

Fig. 2: Our playlist model architecture diagram

In order to make inferences (generate our five recommendations for a given playlist), we give our model a playlist title and song features for the songs in that playlist. We then have our model output a distribution over the vocab size from which we sample five songs. We experimented with different sampling techniques such as top-k sampling and top-p sampling with different amounts of temperature.

C. Training Procedure

1) Loss Function: Our loss function is a specialized version of WARP loss. Traditional WARP loss works in the following way. For each training example (playlist P, positive song i that the model should predict):

1. Compute score (logit) for positive:

$$s_i = s(P, i)$$

2. Sample a negative song j (not in the playlist) at random. Compute score (logit):

$$s_j = s(P, j)$$

3. If $s_j \leq s_i - \text{margin}$, continue sampling negative songs.

4. Stop when you find the first violating negative or some set limit is hit (in our case, 50).

$$s_j > s_i - \text{margin}$$

Define k as the total number of negative samples we had to take.

5. Apply a hinge loss:

$$L = w(k) \cdot \max(0, 1 + s_j - s_i)$$

Where $w(k)$ is the following weight function:

$$w(k) = \sum_{r=1}^{k} \frac{1}{r}$$

Thus, the loss is based on the severity of the violation and if we had to take more negatives to find a violation (our model performed better), the loss will be lower.

In our case, our model's inputs are not the full playlist but rather a proportion of the songs in the playlist. We then have our model make a prediction (an output distribution) for a next song to be added to the playlist it was given. We consider positive samples to be every song that was in the original playlist (both songs that were given to the model and songs that were not) and negative samples as any songs not in the original playlist. We define a "hidden weight" parameter H for the importance of the songs not given to the model and a "seen weight" parameter S for the importance of the songs given to the model. Where

$$H > S$$

so that hidden songs are more important than seen songs.

The loss for a playlist is defined in figure 3 (margin = 1.0).

Take the average of all of these losses to produce a final loss for the playlist L_P.

Therefore, our loss is a hybrid between a sort of reconstruction loss of the songs the model was given and a normal WARP loss that takes into account the model's predictions on missing songs in a playlist. We developed this loss because

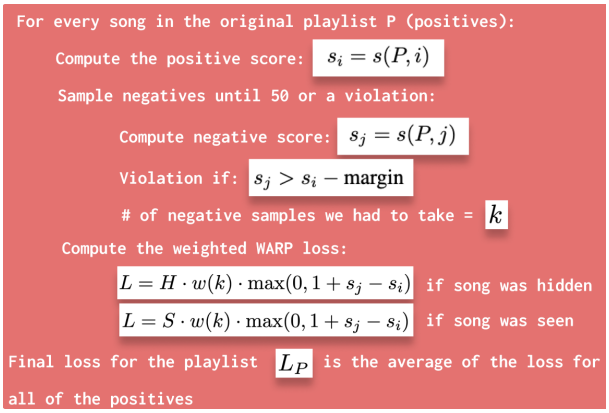

Fig. 3: Custom WARP Loss per playlist

we thought that categorical cross entropy would be too harsh for our model because the space of possible songs that can be predicted is so large. Additionally, categorical cross entropy implies that there is one correct answer for what the next song in a playlist should be and is meant for classification problems. On the other hand, our task is more of a ranking problem where we wanted to encourage our model to learn which kinds of songs fit into a playlist better than others.

2) Optimizer, Batching, and Epochs: The optimizer we used was Adam with a 0.0003 learning rate. This worked really well so we decided not to tune this hyperparameter given our long training times.

Batching was done through first splitting the playlists into buckets based on the number of songs in each playlist. For example, playlists that had between 0 and 20 songs were grouped together, ones with 20-40 songs were grouped together, 40-60, 60-80, 80-120, and 120-200, 200+. These buckets are then batched, where the buckets with longer playlists have a smaller batch size than those with shorter playlists. The batch lengths are as follows: 32,32,32,32,16,8,4. In each batch, we pad playlists so that every playlist in a batch has the length of the longest playlist in the batch. A mask is created to keep track of these fake songs so that we do not use them in our training loop. The reason we bucket is so that we minimize padding which in turn minimizes memory usage as we are not saving an unreasonable amount of padding.

Our model was run for 40 epochs.

3) Hyperparameter Tuning: We focus on tuning hyperparameters related to our loss such as the "seen weight" and "hidden weight" in our loss function. We also experiment with the embedding dimension for the song embeddings and the dimension of the final pooled representation of each playlist. We discuss the results of tuning these parameters in the results section.

D. Evaluation Metrics

To assess our model's performance during training, we use two main metrics. We examine the train and validation loss and we also use R-precision. If a playlist has length R, R-precision is the proportion of songs in the top R predictions of the model that were in the original playlist. We track both a "hidden R-precision" (the proportion of songs that the model was not given as input that were in its top R predictions) to examine how our model is performing at predicting new songs given a playlist and a "total R-precision" to examine how our model is performing at both the reconstruction and prediction aspects of the task.

To assess our recommendation system after training, we manufacture sets of toy examples (playlists with titles and songs) with varying levels of difficulty. For example, easier examples include playlists organized by genre and more difficult examples include playlists organized by a more complex theme such as a "workout" playlist or a "chill slow vibes playlist". We have our system make recommendations for these examples and manually assess their quality.

IV. Results

A. Quantitative Results

1) Ablation Testing: We experiment with the impact of the "seen weight" and "hidden weight" in our loss function. We train our model for 10 epochs on approximately 40% of our dataset for more reasonable training times where the proportion hidden is randomized between 0.5 and 0.1. Table I shows the results of adjusting the "seen weight". By making the "seen weight" smaller, we make the songs that the model is given as input less important in the loss function (put more emphasis on the prediction aspect of the loss rather than the reconstruction aspect). We see inconclusive results on the affect of this hyperparameter.

TABLE I: Ablation Tests for "seen weight" and "hidden weight"

Seen Weight	Hidden Weight	(Best) Validation R-precision (Total)	(Best) Validation R-precision (Hidden)
0.5	1.0	0.224	0.093
0.25	1.0	0.215	0.086
0.10	1.0	0.217	0.089

We also experiment with the impact of the song embedding size (the size of the encoded songs) and the playlist embedding size (the size of the final playlist representation vector). We again train our model for 10 epochs on approximately 40% of our dataset where the proportion hidden is randomized between 0.5 and 0.1. Table II shows the results of adjusting these hyperparameters. Our ablation testing suggests that increasing these embedding sizes negatively impacts performance.

TABLE II: Ablation Tests for "song embedding size" and "playlist embedding size"

Song Embedding Size	Playlist Embedding Size	(Best) Validation R-precision (Total)	(Best) Validation R-precision (Hidden)
128	128	0.224	0.093
128	256	0.223	0.089
256	128	0.207	0.083
256	256	0.103	0.034

2) Training Results: We train our model with seen weight S = 0.5, hidden weight H = 1.0, hiding between 0.5 and 0.1 of the playlists, song embedding size 128 and playlist representation size 128. We achieve performance seen in figures 4, 5, and 6.

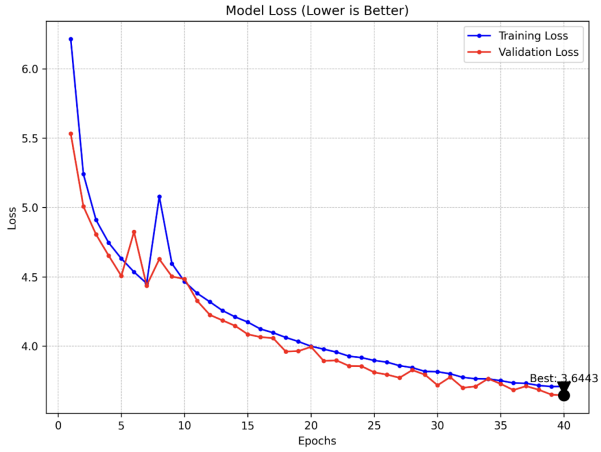

Fig. 4: Loss during training

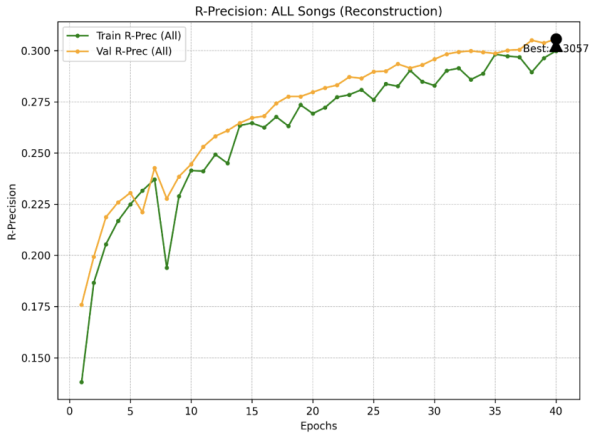

Fig. 5: R-precision (total) during training

B. Qualitative Analysis

Overall, our qualitative results are underwhelming. We create six hand-made playlists that we have our model make recommendations for. Each of these toy example playlists has between 45 and 60 songs. The first three examples are "easier" playlists organized by genre: "Pop", "Rap / Hip-hop", and "American country yehaw". The other three examples are more difficult playlists centered around more complex themes: "Hype up workout", "Chill slow vibes", and "70s hard rock".

Across the board, our model does not appear to give good recommendations that fit in with the vibe of these playlists. Rather, when run with a set seed to ensure determinism, our model appears to exhibit a form of mode collapse often suggesting a similar set of artists and songs regardless of differences in the playlist it is given. Examples of this mode

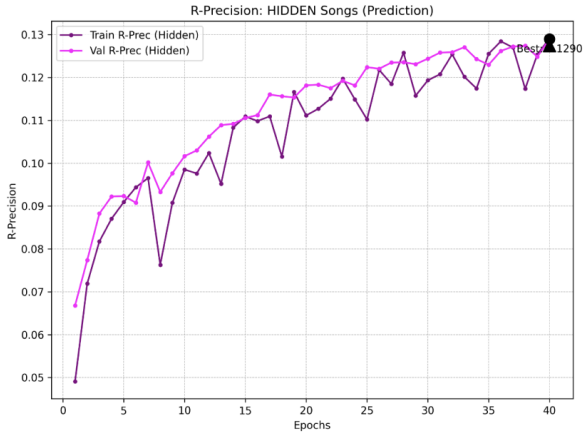

Fig. 6: R-precision (hidden) during training

collapse can be seen in figures 7 and 8 where similar popular songs are recommended for two very different playlists. Results were similar for the more difficult playlists.

→ Pégate (MTV Unplugged Version)[Radio Edit] by Ricky Martin
→ Shining Star by Earth, Wind & Fire
→ 679 (feat. Remy Boyz) by Fetty Wap
→ Intoxicated - New Radio Mix by Martin Solveig;Good Times Ahead
→ Headlights (feat. Ilsey) by Robin Schulz
→ Ghost Town by Adam Lambert
→ False Alarm by Matoma;Becky Hill
→ Move Your Feet by Junior Senior
→ Locked Away (feat. Adam Levine) by R. City;Adam Levine
→ I'm the One (feat. Justin Bieber, Quavo, Chance the Rapper & Lil Wayne) by DJ Khaled;Justin Bieber;Quavo;Chance the Rapper;Lil Wayne

Fig. 7: Recommendations for a "rap / hip-hop" playlist

→ I Don't Like It, I Love It (feat. Robin Thicke & Verdine White) by Flo Rida
→ 679 (feat. Remy Boyz) by Fetty Wap
→ Boom Boom Pow by Black Eyed Peas
→ Pony by Ginuwine
→ Count on Me by Bruno Mars
→ Get Ugly by Jason Derulo
→ Trampoline by Kalin and Myles
→ Shake Senora (feat. T-Pain & Sean Paul) by Pitbull
→ Hey Mama (feat. Nicki Minaj, Bebe Rexha & Afrojack) by David Guetta;Afrojack;Bebe Rexha;Nicki Minaj
→ Partition by Beyoncé

Fig. 8: Recommendations for an "American country yeehaw" playlist

Our qualitative results suggests that the playlists we create and give the model are out of distribution causing the model to fall back on generally popular songs. It also indicates that our title conditioning is not strong enough within our model. This might be due to the fact that it is only the equivalent of one song's encoding in an encoded playlist in our architecture as the title encoding and song encoding are projected to the same size.

C. Goal Achievement

Thus, while we reached our baseline goal of using our proposed architecture to create a system that can make playlist recommendations, we did not meet our target and stretch goals of making a system that can recognize themes of playlists and make good recommendations for them.

V. Challenges

The following are some challenges we encountered during our project:

Preprocessing challenges: During our data collection, we had to strike a balance between practicality and depth of

features. While our dataset of user playlists already grouped Spotify tracks together in playlists, we needed to pair these track IDs with essential audio and song features. Thus, we decided to combine four different datasets, each with song audio information. While the upside of this was a surplus of songs, not all of them contained the same features. Therefore, we decided to implement two optionality columns with the hope that our model can learn when a feature does or does not exist during training.

Runtime challenges: Scarcity in data was far from an issue during this project. However, with such a large amount of playlist-track rows, finding a way to minimize runtime, especially during ablation testing, was crucial. To deal with this issue, we ran our ablations tests with only 10 epochs and reduced our input data to about 40% of the whole dataset. We also utilized the OSCAR computing resources to run our model. Despite using OSCAR, because of the size of our data our runtimes were quite long which hindered our ability to make many changes to our model and test their impact.

VI. Ethics

One of the most important ethical issues for us to consider with our project was bias. Currently, our model is trained on user-made playlists and Spotify song features, which likely over-represents certain genres over others. We also restrict our vocab size to the top K tracks in the dataset, which means that our system implicitly biases towards more popular artists. Thus, it makes sense that our system would be more likely to recommend music by artists who represent cultural majorities and their listening habits. Therefore, this could lead to some listeners who come from minority cultures or who enjoy music from minority cultures having a less satisfying listening experience. Furthermore, artists who make their livelihoods from streaming numbers could be unfairly negatively impacted if their music is not recommended to users because of the biases in our system. Further work could include investigating the biases our system may present more quantitatively by analyzing data on recommendations.

Beyond bias, privacy is another concern. While Spotify wrapped for example is an exciting opportunity for one to learn about their listening habits, it also illustrates how simple it is for a company to gather and extract detailed information about your tendencies. This can lead to revealing unwanted or unrepresentative patterns, and may even become dangerous if the data gets in the wrong hands. Our model however deliberately avoids representing individual users. Whereas most recommendation systems tend to recommend signs based on similar user embeddings (as this makes the task easier), we solely predict tracks based on the audio features of a song. By removing the user-perspective, this can greatly reduce privacy violations.

VII. Reflection

A. Overall Assessment

Overall, our project was very successful as a learning experience. We learned a lot by trying to frame our task as a deep learning problem as we discuss in the Key Takeaways section. However, our model did not work as expected. We created a functioning recommendation system but not a very high quality one. It surprised us that our model was able to perform pretty well on our training data as judged by our metrics but was not able to generalize well to new examples showing how framing the problem correctly and using data suited to the problem are vital to the real-world success of a deep learning project.

B. Evolution and Pivots

We definitely changed our approach as we went along. In the earlier stages of our project, we thought about using more basic architectures and using strategies like clustering for our system before transitioning to a strategy that features more deep learning.

Another pivot we made had to do with our custom loss function. Our first approach involved hiding only one song per playlist and giving the model all but one song, but we realized that giving the model all but one track made our task too close to pure reconstruction risking an ability to generalize past a glorified identity mapping. Therefore, we use a mask to randomly hide a proportion of songs in playlists between 0.5 and 0.1. This ensures that the model does not just learn to reconstruct the playlist and is incentivized to rank unseen songs high as well.

If starting over, we might spend more time inspecting our data and ensuring that it has enough learnable information for our model to work with.

C. Future Work

Future iterations on this project could explore improving our model and recommendation system in a few ways. First, audio files could be used as a feature in our model. This could be done in the form of a sonogram or .wav file where a CNN could be used to understand the audio features. Additionally, in our model we do not utilize the text data of the artist name, song name, and album name for a given song. These text features could be embedded with a pre-trained text encoder like USE so that our model could learn more relationships that could help it to recommend songs for playlists. Another helpful exploration could be investigating the impact of the margin and number of negative sample hyperparameters in our loss function. Furthermore, adjusting the architecture to try to have the title embedding carry more weight in the model's predictions would be valuable. Lastly, the mode collapse we see in our model's predictions could be examined more thoroughly. For example, the output distributions could be looked at to inform why this mode collapse is happening in order to inform further improvements.

D. Key Takeaways

Despite only modestly successful results, this project was a valuable learning experience for us. First, working as a team and bouncing ideas off of each other was very valuable especially in the early brainstorming stages where we took

a while to converge to a topic that we all liked. However, once we landed on our topic we were all motivated to work on it. As far as technical skills, we took away a number of learnings. First, we gained experience abstracting transformer architectures to a domain outside of natural language processing, an endeavor that required us to think more deeply about assumptions traditional transformers make like the use of positional encodings and vocabulary sizes. Second, we really embraced the modularity of deep learning models by using pre-trained models, MLPs, and attention layers throughout our architecture. Third, we experimented with a more specialized loss function. We were able to recognize that more traditional loss functions we had learned about in the course would not apply very well to our use case so we were able to learn about ranking-based loss functions (WARP) and even experimented with modifying it to include some ideas we had learned about such as the idea of a reconstruction loss. Lastly, we all gained experience using remote computing resources to train our model.

VIII. Conclusion

To conclude, we jump off of previous literature to use set transformers for the task of playlist recommendation and develop a custom loss function that we believe is well-suited to the task. While our recommendation system does not work very well, our model is able to train well and perform well on our metrics indicating that with the right data and hyperparameter tuning, our model could be used effectively for the task of playlist recommendation.

Acknowledgments

We would love to thank our TA mentor Armaan Patankar for his support and advice and the entire CSCI 1470 staff and Professor Eric Ewing for a great semester.

References

[1] A. Ferraro, D. Bogdanov, J. Yoon, K. Kim, and X. Serra, "Automatic playlist continuation using a hybrid recommender system combining features from text and audio," in *Proceedings of the ACM Recommender Systems Challenge 2018*, 2018, pp. 1–5.

[2] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 3744–3753.

[3] C.-W. Chen, P. Lamere, M. Schedl, and H. Zamani, "Recsys challenge 2018: Automatic music playlist continuation," in *Proceedings of the 12th ACM Conference on Recommender Systems*, 2018, pp. 527–528.

[4] https://www.kaggle.com/datasets/rodolfofigueroa/spotify-12m-songs.

[5] https://www.kaggle.com/datasets/amitanshjoshi/spotify-1million-tracks.

[6] https://www.kaggle.com/datasets/yamaerenay/spotify-dataset-19212020-600k-tracks?select=tracks.csv.

[7] https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset/data.

[8] D. Cer, Y. Yang, S. Kong, N. Hua, N. Limtiaco, R. S. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, Y. Sung, B. Strope, and R. Kurzweil, "Universal sentence encoder," *CoRR*, vol. abs/1803.11175, 2018. [Online]. Available: http://arxiv.org/abs/1803.11175